Session Type State Spaces Form Lattices

Anonymous
Anonymous
anonymous@example.org

We prove that the state space of every well-formed session type, quotiented by strongly connected components, forms a bounded lattice. The proof proceeds by structural induction on the session type: **end** gives a trivial lattice; branch and selection create shared extrema serving as meets and joins; recursion and cyclic named definitions introduce cycles absorbed by the SCC quotient; and n-ary parallel composition yields an n-dimensional product lattice. We prove that duality (swapping branch and selection) preserves state-space isomorphism—polarity is invisible at the lattice level—and derive a concrete application: Gay–Hole width subtyping corresponds to lattice embedding, with the selection case following from the branch case via duality. We validate this result with two independently developed tools and 108 benchmark protocols drawn from networking, databases, distributed systems, AI, and fault tolerance. All 108 are confirmed to form lattices—93 (86%) distributive, 15 (14%) non-distributive—and we mechanically verify the core proof in Lean 4 (14 modules, 2,564 lines, all constructor lemmas fully proved).

1 Introduction

Session types [12, 26] describe the communication protocols that govern interaction with objects, channels, or services. A session type specifies the allowed sequences of operations—method calls, branches, selections—together with their ordering constraints. The *state space* of a session type is the labeled transition system (LTS) obtained by "executing" the type: each state represents a protocol stage, and each transition a permitted action.

State spaces are routinely used for verification, subtype checking, and protocol analysis, yet their algebraic structure has received little attention. Gay and Hole [10], for instance, study subtyping as a preorder on session types but do not investigate the structure of the underlying state spaces. In this paper we show that *the state space of every well-formed session type, after quotienting by strongly connected components, is a bounded lattice*. We call this lattice a *reticulate*—from Latin *reticulatus*, the Romance-language root of "lattice" (Portuguese *reticulado*, French *réticulé*), giving session type lattices their own name while remaining etymologically precise. The lattice structure arises naturally from the session type constructors: branch (&) and selection ($\oplus$) create shared extrema that serve as meets and joins; sequencing prepends a new maximum element; recursion introduces cycles absorbed by the SCC quotient; and n-ary parallel composition ($\|$) yields n-dimensional product lattices.

The $\|$ constructor is where lattice structure becomes *necessary* rather than merely convenient. Without $\|$, the SCC quotient of a session type's state space is a tree (pure sequencing), a diamond (binary choice), or a DAG (nested choice). With $\|$, the n-fold product of state spaces inherently has meets and joins corresponding to fork and synchronisation points of concurrent execution. We define $\|$ as natively n-ary (not binary with a nesting restriction), yielding n-dimensional product lattices directly.

Our contributions are: (1) a proof that the SCC quotient of every well-formed session type's state space is a bounded lattice, by structural induction; (2) a proof that duality preserves state-space isomorphism (Section 6); (3) a concrete application: Gay–Hole width subtyping corresponds to lattice

Submitted to:
ICE 2026

embedding, with the selection case derived from the branch case via duality (Section 7); (4) empirical validation on 108 benchmark protocols using two independently developed tools; (5) mechanised verification of the proof in Lean 4 (14 modules, zero unfinished proofs); (6) a distributivity classification identifying the two structural patterns (N_5 from polarity nesting, M_3 from wide branching) that account for all non-distributive cases.

Sections 2 and 3 recall background and define the state-space construction. Section 4 presents the proof; Section 5 works through a complete example. Sections 6 and 7 develop the duality and subtyping applications. Sections 8 and 9 report empirical results and tool support. Section 10 discusses related work and Section 11 concludes.

2 Preliminaries

2.1 Session Types

We consider session types for objects [11, 27], describing the permitted sequences of method calls on a typed object:

$$
\begin{array}{rcll}
S & ::= & \&\{m_1 : S_1,\ \ldots,\ m_n : S_n\} & \text{(branch, } n \geq 0) \\
 & \mid & \oplus\{l_1 : S_1,\ \ldots,\ l_n : S_n\} & \text{(selection, } n \geq 0) \\
 & \mid & (S_1 \parallel \cdots \parallel S_n) & \text{(parallel, } n \geq 2) \\
 & \mid & \mu X.S & \text{(recursion)} \\
 & \mid & X & \text{(variable / name)} \\
 & \mid & \textbf{end} & \text{(terminated)} \\
P & ::= & S\ \textbf{where}\ X_1 = S_1,\ \ldots,\ X_k = S_k & \text{(program, } k \geq 0)
\end{array}
$$

A program P is a root type S with $k \geq 0$ supporting named definitions; when $k = 0$ the **where** clause is omitted and P reduces to a bare type. Each S_i may reference any name X_j; a program is *well-defined* if every free name is among $\{X_1, \ldots, X_k\}$. Named definitions serve as a *top-level protocol design* mechanism: the designer decomposes a protocol into named, reusable phases for clarity and modularity—the standard representation in typestate tools [16] and automata theory. Names need not be recursive: they may simply alias sub-protocols to avoid repetition and reduce errors in large specifications. When names do form reference cycles, the system naturally expresses recursion (including mutual recursion), subsuming $\mu X.S$ as the special case $\{X = S\}$.

The grammar thus has seven constructs (six for types, one for programs), three of which are n-ary (branch, selection, parallel). We now describe each constructor in turn.

Branch and selection. The two choice constructors are duals in the sense of linear logic: $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ is an *external* choice, where the environment picks which method to invoke, and $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$ is an *internal* choice, where the typed object itself picks a label (typically the outcome of a method, such as an enum tag or a success/error branch). Both admit $n = 0$ as honest duals: $\&\{\}$ denotes a state with no enabled methods (no external choice offered) and $\oplus\{\}$ denotes a state with no labels chosen (no internal choice made). The two empty cases yield the same 2-element state space $\{q_0, q_\bot\}$ under their respective polarities and serve as canonical deadlock states. Forcing the asymmetric restriction $\oplus\{\}, n \geq 1$ would break branch/selection duality at the boundary, so we admit both.

Parallel composition. Parallel composition $(S_1 \parallel \cdots \parallel S_n)$ models n-way concurrent access to a shared object: each branch S_i executes independently, and the composite terminates only once every branch has reached **end**. We take parallel as natively n-ary rather than binary with nested associativity, because the n-fold interleaving of method calls admits no canonical binary parenthesisation and because the resulting state space is the n-fold product lattice $\mathscr{L}(S_1) \times \cdots \times \mathscr{L}(S_n)$ under componentwise ordering—an n-dimensional grid whose structure a binary encoding with a nesting restriction cannot express. Making parallel n-ary simplifies the well-formedness conditions (**WF-Par**, Definition 2): each branch must terminate and no recursion variable may cross branch boundaries, with no further restriction on nesting depth because there is no nesting.

Recursion and variables. The explicit recursion constructor $\mu X.S$ binds a name X whose free occurrences inside S refer to the whole type, allowing protocols to loop (e.g., a file that can be read repeatedly). A bare variable X is either bound by an enclosing $\mu X.\cdot$ or by a top-level named definition in the program's **where** clause; in both cases it is a reference, not an action. Recursion is required to be *guarded* (at least one method constructor on every path from the binder back to the variable, Definition 2), which makes the set of reachable states finite and rules out degenerate types such as $\mu X.X$. Because named definitions subsume recursion as the special case $\{X = S\}$, the two mechanisms coexist and neither imposes an expressiveness ceiling on the other—the choice between them is purely one of protocol design.

Termination. The constant **end** denotes explicit protocol termination: a transition into the canonical bottom state $q_\perp$ after which no further action is possible. **end** is retained as a distinct primitive even though it looks like the empty branch $\&\{\}$: the empty branch represents a state with no offered methods, which need not be a terminal state of the whole session (a parallel branch that has finished its own actions is not "done" until its siblings finish too), whereas **end** unambiguously signals completion. Every terminated session is an empty branch, but not every empty branch is terminated.

Remark 1 (Syntactic sugar). Sequencing $m\,.\,S$ abbreviates the single-method branch $\&\{m : S\}$. For example, a file object: $open\,.\,\mu X.\&\{read : \oplus\{data : X,\ eof : \&\{close : \mathbf{end}\}\}\}$.

Definition 2 (Well-formedness)**.** A session type (or program) is *well-formed* if all of the following hold.

Guardedness. Every recursion $\mu X.S$ is *guarded*: at least one constructor separates $\mu X.\cdot$ from each occurrence of X in S.

Generalised guardedness. In equation systems, every cyclic reference path passes through at least one constructor (guardedness generalised to named definitions).

WF-Par. For each parallel $(S_1 \parallel \cdots \parallel S_n)$, both of the following hold: (i) every branch S_i is *terminating* (every maximal path reaches **end**); (ii) no recursion variable bound in one branch is free in another.

Guardedness and its generalisation to named definitions are standard [12, 26, 16]; **WF-Par** is specific to our setting and is the minimal condition that makes the product construction yield a bounded lattice (Corollary 19 shows it is tight). **WF-Par** ensures that all parallel branches eventually reach a common termination point, and that the product construction inherits SCC structure componentwise. Because parallel is natively n-ary, no restriction on nesting is needed: n-way products are handled directly.

Scope. The grammar targets session types for *objects*—method calls on shared state—and deliberately omits channel-specific operations (send/receive, delegation) found in process calculi [12]. **WF-Par** restricts parallel composition to terminating branches with independent variables; relaxing these conditions (e.g., non-terminating servers) is an open problem. Like all related work [10, 16], we consider finite-state (regular) session types; extending to context-free types would require different algebraic machinery.

2.2 Lattice Theory

Definition 3 (Bounded lattice)**.** A *bounded lattice* is a poset $(L, \leq)$ with least element $\bot$ and greatest element $\top$ such that every pair of elements has a *meet* (greatest lower bound) $a \wedge b$ and a *join* (least upper bound) $a \vee b$.

Definition 4 (Product lattice)**.** Given bounded lattices $(L_1, \leq_1), \ldots, (L_n, \leq_n)$, their *product lattice* is $L_1 \times \cdots \times L_n$ with componentwise ordering: $(a_1, \ldots, a_n) \leq (b_1, \ldots, b_n)$ iff $a_i \leq_i b_i$ for all i. Meets, joins, top, and bottom are componentwise. The product of finitely many bounded lattices is a bounded lattice.

A bounded lattice is *distributive* if $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ for all elements; equivalently (Birkhoff [2]), if it contains no sublattice isomorphic to N_5 (the pentagon) or M_3 (the diamond with three atoms).

A subset D of a poset is *upward-closed* if $x \in D$ and $x \leq y$ implies $y \in D$; dually, *downward-closed* (an *order-ideal*).

A *strongly connected component* (SCC) of a directed graph is a maximal set of mutually reachable vertices; the *SCC quotient* collapses each SCC to a single node, yielding a DAG [23].

3 State-Space Construction

Definition 5 (State space)**.** A state space is a tuple $\mathscr{L}(S) = (Q, \Sigma, \delta, q_\top, q_\bot)$ where Q is a finite set of states, Σ is a finite alphabet of action labels, $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation, $q_\top \in Q$ is the initial state, and $q_\bot \in Q$ is the terminal state.

The construction is defined by structural induction on S:

end. A single state, both initial and terminal.

Branch $\&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$**.** Build each $\mathscr{L}(S_i)$, identifying all terminal states into a shared $q_\bot$. Add a fresh initial q_0 with transitions $q_0 \xrightarrow{m_i} q_\top^{S_i}$. Sub-state-spaces are disjoint above $q_\bot$.

Selection $\oplus\{l_1{:}S_1, \ldots, l_n{:}S_n\}$**.** Identical to branch at the state-space level: the construction depends only on the set of continuations, not on who makes the choice. The polarity distinction (external choice for & vs. internal choice for $\oplus$) is relevant for subtyping [10] and duality, but not for the reachability structure that determines the lattice.

Recursion $\mu X.S$**.** Build $\mathscr{L}(S)$, allocating a placeholder state for each occurrence of X. Then redirect all transitions targeting a placeholder to the initial state $q_\top^S$, creating back-edges (cycles), and remove the placeholder states.

Named definitions $P = S$ **where** $X_1 = S_1, \ldots, X_k = S_k$**.** Build $\mathscr{L}(S)$ and each $\mathscr{L}(S_i)$ $(1 \leq i \leq k)$, allocating a fresh placeholder state $p_{i,j}$ for every occurrence of name X_j in S_i (and similarly for occurrences of names in the root S). Then perform a single global redirection: for each name X_j, replace every transition $q \xrightarrow{a} p_{*,j}$ by $q \xrightarrow{a} q_\top^{S_j}$, and remove all placeholder states. The result is a single connected state space with entry point $q_\top^S$. Acyclic references (no cycle through the name graph) are effectively inlined; cyclic references create back-edges, which the SCC quotient absorbs exactly as in the single-variable recursion case.

Parallel $(S_1 \parallel \cdots \parallel S_n)$**.** See Definition 9.

Definition 6 (Reachability ordering)**.** Write $s_1 \twoheadrightarrow s_2$ ("s_1 reaches s_2") iff there exists a path from s_1 to s_2 in (Q, δ). The relation $\twoheadrightarrow$ is a preorder; cycles from recursion prevent antisymmetry. The *SCC quotient* $\mathscr{L}(S)/\equiv$ is the canonical antisymmetric quotient: it identifies mutually reachable states (those in the same protocol phase), yielding the unique partial order induced by $\twoheadrightarrow$. We define $[s_1] \leq [s_2]$ iff $s_2 \twoheadrightarrow s_1$: a state is *smaller* when less protocol remains. The terminal $q_\bot$ is least, the initial $q_\top$ is greatest, and execution descends from $\top$ to $\bot$.

Proposition 7 (Unique extrema)**.** *For every well-formed S, $\mathscr{L}(S)$ has a unique initial state $q_\top$ that reaches all states, and a unique terminal state $q_\bot$ reachable from all states.*

Proof. By induction on the construction: each case introduces $q_\top$ and $q_\bot$ (End: single state; choice: fresh $q_\top$ plus shared $q_\bot$; recursion: back-edges preserve extrema in the quotient; parallel: componentwise extrema). □ □

Lemma 8 (Polarity erasure)**.** *For any labels $a_1, \ldots, a_n$ and well-formed types $S_1, \ldots, S_n$,*

$$\mathscr{L}(\&\{a_1{:}S_1, \ldots, a_n{:}S_n\}) \;=\; \mathscr{L}(\oplus\{a_1{:}S_1, \ldots, a_n{:}S_n\}).$$

Proof. Immediate from the construction: both create a fresh $q_\top$ with n transitions to sub-state-space entries, sharing $q_\bot$. Polarity is a typing discipline, not a structural property of the state space. □ □

Definition 9 (Product construction)**.** Given $\mathscr{L}(S_1), \ldots, \mathscr{L}(S_n)$, define

$$\mathscr{L}(S_1 \parallel \cdots \parallel S_n) = (Q_1 \times \cdots \times Q_n,\ \textstyle\bigcup_i \Sigma_i,\ \delta_\times,\ (q^1_\top, \ldots, q^n_\top),\ (q^1_\bot, \ldots, q^n_\bot))$$

where $\delta_\times$ advances one component at a time: $(s_1, \ldots, s_i, \ldots, s_n) \xrightarrow{a} (s_1, \ldots, s'_i, \ldots, s_n)$ for each $s_i \xrightarrow{a} s'_i$ in δ_i. By **WF-Par**(ii), the label sets Σ_i are pairwise disjoint, so no label is shared between branches.

4 The Lattice Theorem

We prove that the SCC quotient of every well-formed session type's state space is a bounded lattice. The proof proceeds by structural induction on S, with one lemma per constructor. We write $\leq$ for the partial order on the quotient $\mathscr{L}(S)/\equiv$. Variables X are not a separate case: they serve as placeholders resolved by the recursion or named-definitions construction (Section 3) and never appear as standalone state spaces in a well-formed program.

4.1 Base Cases

Lemma 10 (End)**.** *$\mathscr{L}(\mathbf{end})/\equiv$ is a bounded lattice.*

Proof. The state space $\mathscr{L}(\mathbf{end})$ consists of a single state q that is both initial and terminal: $q_\top = q_\bot = q$. The SCC quotient $\mathscr{L}(\mathbf{end})/\equiv$ is the one-element poset $\{[q]\}$, which satisfies all lattice axioms: $[q]$ is both top and bottom, and $[q] \wedge [q] = [q] \vee [q] = [q]$. □

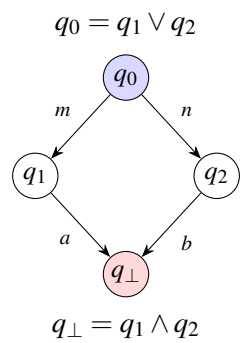

Figure 1: State space of $\&\{m : a\,.\,\mathbf{end},\ n : b\,.\,\mathbf{end}\}$: the diamond lattice M_2. The initial state is the join, the terminal state the meet, of q_1 and q_2.

4.2 Choice Constructors

Branch and selection have identical state-space structure (Section 3), so we treat them uniformly as $C\{a_1 : S_1, \ldots, a_n : S_n\}$.

Lemma 11 (Choice)**.** *If $\mathscr{L}(S_i)/\equiv$ is a bounded lattice for each i, then $\mathscr{L}(C\{a_1 : S_1, \ldots, a_n : S_n\})/\equiv$ is a bounded lattice.*

Proof. The construction creates a fresh initial state q_0 with transitions to each sub-state-space entry, and a shared terminal $q_\perp$. For states in the same sub-lattice, meets and joins exist by induction. For states $a \in \mathscr{L}(S_i)$ and $b \in \mathscr{L}(S_j)$ $(i \neq j)$: the sub-state-spaces are disjoint above $q_\perp$, so any upper bound of $\{a,b\}$ must reach both sub-spaces and therefore must be q_0 (the only such state); hence $a \vee b = q_0$. Dually, any lower bound of $\{a,b\}$ must be reachable from both, hence must lie at or below $q_\perp$; since $q_\perp$ is least, $a \wedge b = q_\perp$. See Figure 1 for an illustration. □ □

Remark 12 (Sequencing). The syntactic sugar $m\,.\,S \equiv \&\{m : S\}$ (Remark 1) is the $n = 1$ case of Lemma 11: the fresh $q_\top$ has a single transition to $q_\top^S$, yielding a chain that prepends one element above the existing lattice. No separate lemma is needed.

4.3 Recursion

Recursion is the most delicate case: back-edges create cycles that cause the SCC quotient to merge classes.

Lemma 13 (Top absorption)**.** *Let $(L, \leq)$ be a bounded lattice and $D \subseteq L$ an upward-closed set containing $\top$. Define $L' = (L \setminus D) \cup \{\top\}$ with the restricted ordering. Then L' is a bounded lattice.*

Proof. Since D is upward-closed, $\perp \notin D$, so L' retains both extrema. For $a, b \in L \setminus D$: the meet $a \wedge_L b$ lies in $L \setminus D$ (the complement of an upward-closed set is downward-closed), so it is preserved. For the join: if $a \vee_L b \in L \setminus D$, it is preserved. If $a \vee_L b \in D$, then $a \vee_{L'} b = \top$: any non-$\top$ upper bound $u \in L \setminus D$ would satisfy $u \geq a \vee_L b \in D$, placing $u \in D$ since D is upward-closed—contradicting $u \in L \setminus D$. □ □

Lemma 14 (Recursion)**.** *If $\mathscr{L}(S)/\equiv$ is a bounded lattice, then $\mathscr{L}(\mu X\,.\,S)/\equiv$ is a bounded lattice.*

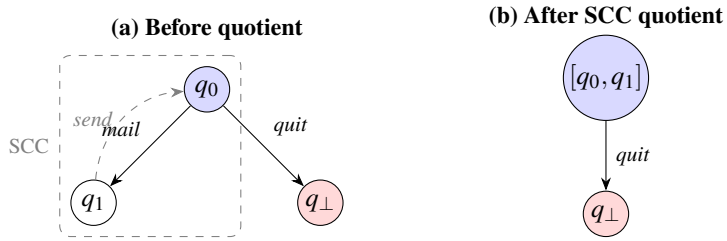

Figure 2: SMTP session type $\mu X.\&\{mail : \&\{send : X\},\ quit : \mathbf{end}\}$. (a) Before quotient: the recursion redirects *send* from the placeholder to q_0, creating a cycle $q_0 \to q_1 \to q_0$ (dashed). (b) After SCC quotient: the cycle collapses into $[q_0, q_1]$, yielding a 2-element chain. Verified by RETICULATE: 3 states, 3 transitions, SCC $= \{q_0, q_1\}$.

Proof. Adding back-edges from each placeholder of X to the initial state $q_\top^S$ makes these states mutually reachable with $q_\top^S$, hence they are collapsed into the same equivalence class $[q_\top^S]$. The set D of quotient classes absorbed into $[q_\top^S]$ is upward-closed: if $[s] \in D$ and $[s] \leq [r]$, then $r \twoheadrightarrow s$ (since $[r] \geq [s]$), $s \twoheadrightarrow q_\top^S$ (since $[s] \in D$), and $q_\top^S \twoheadrightarrow r$ (as initial state); so r and $q_\top^S$ are mutually reachable, giving $[r] \in D$. By Lemma 13, collapsing D into a single top element yields a bounded lattice. □ □

Lemma 15 (Named definitions)**.** *If each $\mathscr{L}(S_i)/\equiv$ in an equation system $P = \{X_1 = S_1, \ldots, X_k = S_k\}$ is a bounded lattice, then $\mathscr{L}(P)/\equiv$ is a bounded lattice.*

Proof. The construction connects the k named sub-state-spaces by redirecting placeholder transitions for each name X_j to $q_\top^{S_j}$. Non-recursive references (acyclic names) are effectively inlined and preserve lattice structure trivially. Cyclic references create back-edges that merge states into SCCs, exactly as in the single-recursion case. The set of quotient classes absorbed into each SCC is upward-closed by the same argument as Lemma 14. Applying Lemma 13 iteratively (once per SCC created by cyclic references) yields a bounded lattice; the order of absorption is immaterial since each step preserves the lattice property and the SCCs are determined by the final reachability graph, not by the order of construction. Since the SCC quotient depends only on the reachability graph—not on how names are organised—the resulting quotient lattice is identical for any encoding of the same finite-state protocol. □ □

Example 16 (Top absorption on SMTP). Consider the SMTP session type (Figure 2):

$$S_{smtp} = \mu X.\&\{mail : \&\{send : X\},\ quit : \mathbf{end}\}$$

The body is a branch with $n = 2$. Before recursion, the state space has three states: q_0 (branch entry), q_1 (after *mail*, before *send*), and $q_\perp$ (terminal). The placeholder for X is the target of the *send* transition from q_1.

The recursion redirects the *send* transition to the body's initial state: $q_1 \xrightarrow{send} q_0$. The placeholder is removed. Now q_0 and q_1 are mutually reachable ($q_0 \xrightarrow{mail} q_1 \xrightarrow{send} q_0$), forming an SCC. By Lemma 13, the upward-closed set $D = \{q_0, q_1\}$ collapses into a single class $[q_0, q_1]$. Since $q_\perp \notin D$ (it is minimal), the quotient retains both extrema: the result is a 2-element chain $[q_0, q_1] > q_\perp$—a bounded lattice. Verified by RETICULATE: 3 states, 3 transitions, SCC $= \{q_0, q_1\}$.

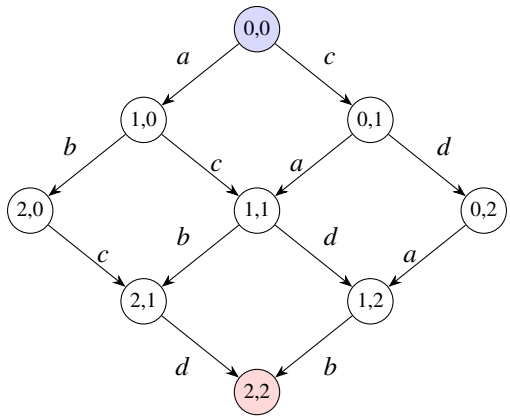

Figure 3: Product lattice $\mathscr{L}(\&\{a : \&\{b : \mathbf{end}\}\} \parallel \&\{c : \&\{d : \mathbf{end}\}\})$. The 9 states form a 3×3 grid lattice with componentwise ordering.

4.4 Parallel Composition

Lemma 17 (Parallel)**.** *If $\mathscr{L}(S_i)/{\equiv}$ is a bounded lattice for each $i \in \{1, \ldots, n\}$, then $\mathscr{L}(S_1 \parallel \cdots \parallel S_n)/{\equiv}$ is a bounded lattice.*

Proof. By Definition 9 and the independence of the n branches (ensured by **WF-Par**: all branches terminate, no shared variables), the product inherits SCC structure componentwise—an SCC in the product is a product of SCCs from each component:

$$\mathscr{L}(S_1 \parallel \cdots \parallel S_n)/{\equiv} \;\cong\; \textstyle\prod_{i=1}^{n} (\mathscr{L}(S_i)/{\equiv_i}).$$

The product of finitely many bounded lattices is a bounded lattice (meets and joins are componentwise). See Figure 3 for an illustration with $n = 2$. □ □

4.5 Main Theorem

Theorem 18 (Reticulate Theorem)**.** *For every well-formed session type S (including equation systems), $\mathscr{L}(S)/{\equiv}$ is a bounded lattice.*

Proof. By structural induction on S, applying one lemma per constructor. The base case $S = \mathbf{end}$ is Lemma 10. Choice $S = \&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$ or $S = \oplus\{l_1{:}S_1, \ldots, l_n{:}S_n\}$ is Lemma 11, creating shared extrema across disjoint sub-lattices (the $n = 1$ case subsumes sequencing; see Remark 12). Recursion $S = \mu X.S'$ is Lemma 14, where back-edges are absorbed by the SCC quotient via the top-absorption lemma (Lemma 13). Named definitions $P = \{X_1 = S_1, \ldots, X_k = S_k\}$ are Lemma 15, covering both aliases and cyclic references. Parallel composition $S = (S_1 \parallel \cdots \parallel S_n)$ is Lemma 17, yielding an n-fold product lattice. Each case preserves the bounded lattice property, completing the induction. □

Corollary 19 (WF-Par tightness)**.** ***WF-Par*** *is tight: relaxing either of its two conditions yields types whose SCC-quotiented state spaces are not lattices.*

Example 20 (WF-Par counterexamples). We give one counterexample per WF-Par condition.

(i) *Non-terminating branch.* $(\mu X.\&\{a : X\}) \parallel \&\{b : \mathbf{end}\}$: the left branch loops forever, so the product has no state where both components have reached **end**. The lattice axiom that fails is the existence of $\bot$—there is no terminal state reachable from all product states.

(ii) *Shared variables.* $\mu X.(\&\{a : X\} \parallel \&\{b : X\})$: the recursion variable X is free in both branches, so the back-edge from the placeholder merges the two components' SCCs non-trivially. After quotient, the states $(q_a, q_\bot)$ and $(q_\bot, q_b)$ have no least upper bound: both (q_a, q_b) and the collapsed SCC class are upper bounds, but neither is below the other.

Both counterexamples are verified in the Lean 4 mechanisation (`Counterexample.lean`, 119 lines). Note that no restriction on nesting is needed: with n-ary parallel, $(\&\{a : \mathbf{end},\ b : \mathbf{end}\} \parallel \&\{c : \mathbf{end}\} \parallel \&\{d : \mathbf{end},\ e : \mathbf{end}\})$ is a single 3-way product of bounded lattices, hence a bounded lattice.

Proposition 21 (Necessary conditions for realisability)**.** *If a finite bounded lattice L is the reticulate $\mathscr{L}(S)/\equiv$ of some well-formed session type S, then: (i) $\top$ has out-degree ≥ 1 in the Hasse diagram (from the outermost constructor); (ii) every non-extremal element is either a* join node *(reached from $\top$ by a unique path of choice constructors) or a* product element *(a tuple from a $\parallel$ sub-lattice); and (iii) every product sub-lattice is a direct product of reticulates. In particular, the free modular lattice on three generators is not a reticulate.*

5 Worked Example: The File Object

We trace the lattice proof on the File Object protocol from Remark 1:

$$S_{file} = \&\{open : \mu X.\&\{read : \oplus\{data : X,\ eof : \&\{close : \mathbf{end}\}\}\}\}$$

This example exercises four of the six constructor lemmas: End, Choice (four times, including one single-method branch for the sequencing sugar), and Recursion. Figure 4 shows the final Hasse diagram.

Step 1: innermost base case. The innermost subterm is **end**. By Lemma 10, $\mathscr{L}(\mathbf{end})/\equiv$ is the trivial one-element lattice $\{q_\bot\}$.

Step 2: close branch. $\&\{close : \mathbf{end}\}$ adds a fresh top q_3 with transition $q_3 \xrightarrow{close} q_\bot$. By Lemma 11 with $n = 1$, this is a 2-element chain $q_3 > q_\bot$—trivially a lattice.

Step 3: selection. $\oplus\{data : X,\ eof : \&\{close : \mathbf{end}\}\}$ applies Lemma 11 with $n = 2$. A fresh top q_2 is added with transitions $q_2 \xrightarrow{data} q_X$ (placeholder for X) and $q_2 \xrightarrow{eof} q_3$. The two sub-state-spaces are disjoint above the shared $q_\bot$, forming a diamond: q_2 is the join and $q_\bot$ the meet of q_X and q_3.

Step 4: read branch. $\&\{read : \ldots\}$ applies Lemma 11 with $n = 1$, prepending a fresh top q_1 via $q_1 \xrightarrow{read} q_2$. The result is a 5-element lattice (chain from q_1 to q_2, then diamond below).

Step 5: recursion. $\mu X.\ldots$ applies Lemma 14. The recursion redirects all transitions targeting the placeholder q_X to the body's initial state q_1, creating a back-edge $q_2 \xrightarrow{data} q_1$ (was $q_2 \to q_X$). The placeholder q_X is removed. Now q_1 and q_2 are mutually reachable ($q_1 \xrightarrow{read} q_2 \xrightarrow{data} q_1$), forming an SCC. By Lemma 13, the upward-closed set $D = \{q_1, q_2\}$ collapses into a single class $[q_1, q_2]$ serving as the new top of the sub-lattice. The quotient has 3 states below $[q_1, q_2]$: just q_3 and $q_\bot$, giving a 3-element chain $[q_1, q_2] > q_3 > q_\bot$—trivially a lattice.

Step 6: open branch. Finally, $\&\{open : \ldots\}$ applies Lemma 11 with $n = 1$, adding q_0 as the new top. The final lattice has 4 equivalence classes forming a chain: $q_0 > [q_1, q_2] > q_3 > q_\bot$—a bounded

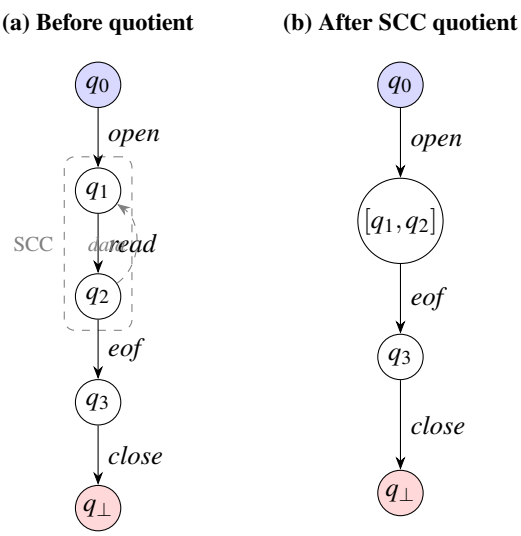

Figure 4: File Object protocol $\&\{open : \mu X.\&\{read : \oplus\{data : X,\ eof : \&\{close : \mathbf{end}\}\}\}\}$. (a) Before quotient: the recursion redirects *data* from the placeholder to q_1, creating a cycle $q_1 \to q_2 \to q_1$ (dashed). (b) After SCC quotient: the cycle collapses into $[q_1, q_2]$, yielding a 4-element chain. Verified by RETICULATE: 5 states, 5 transitions, SCC $= \{q_1, q_2\}$, quotient is a bounded lattice.

lattice. This 4-element chain illustrates that even a simple recursive protocol produces a non-trivial SCC quotient. The SCC $[q_1, q_2]$ captures the *read/data loop*: the protocol can cycle between the read state and the selection state indefinitely before choosing *eof*.

Each step preserves the lattice property by its corresponding lemma, and the composition of all steps proves $\mathscr{L}(S_{file})/\equiv$ is a bounded lattice by Theorem 18.

Example 22 (Parallel: read/write). Consider a shared buffer with concurrent read and write access: $S_{buf} = (\&\{read : \mathbf{end}\} \parallel \&\{write : \mathbf{end}\})$. By Lemma 17, the state space is the product of two 2-element chains: $\mathscr{L}(S_{buf}) = \{(0,0), (0,1), (1,0), (1,1)\}$ — a 2×2 grid lattice (the Boolean lattice B_2) with $\top = (0,0)$ (both actions pending) and $\bot = (1,1)$ (both completed). The four elements are totally ordered on neither component, so the lattice has two incomparable elements $(0,1)$ and $(1,0)$—this is where the $\parallel$ constructor makes lattice structure *necessary*, as the meet and join of these elements are respectively $\bot$ and $\top$. RETICULATE confirms: 4 states, 4 transitions, distributive lattice.

6 Duality Preserves Lattice Isomorphism

The *dual* of a session type swaps branch and selection: $dual(\&\{m_i : S_i\}) = \oplus\{m_i : dual(S_i)\}$ and vice versa; all other constructors (**end**, $\parallel$, $\mu X.\cdot$) are covariant. Duality is an involution: $dual(dual(S)) = S$.

Proposition 23 (Duality isomorphism)**.** *For every well-formed session type S,*

$$\mathscr{L}(S)/\equiv \ \cong\ \mathscr{L}(dual(S))/\equiv$$

as bounded lattices.

Proof. By structural induction on S.

Base case ($S = \mathbf{end}$): $dual(\mathbf{end}) = \mathbf{end}$, so the state spaces are identical.

Choice ($S = \&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$): $dual(S) = \oplus\{m_1{:}dual(S_1), \ldots, m_n{:}dual(S_n)\}$. By the construction in Section 3, branch and selection produce identical state spaces: both create a fresh $q_\top$ with n transitions to sub-state-space entries, sharing $q_\bot$. The construction depends only on the set of continuations, not on the polarity. By induction, $\mathscr{L}(S_i)/{\equiv} \cong \mathscr{L}(dual(S_i))/{\equiv}$ for each i. The n sub-isomorphisms, together with the identity on $q_\top$ and $q_\bot$, yield an isomorphism of the whole choice state space.

Recursion ($S = \mu X.S'$): $dual(\mu X.S') = \mu X.dual(S')$. The recursion construction redirects placeholder transitions to the body's initial state. Since dual acts on the body's constructors (swapping & and $\oplus$) but preserves the placeholder and entry-state structure, the same back-edges are created in both $\mathscr{L}(\mu X.S')$ and $\mathscr{L}(\mu X.dual(S'))$, yielding isomorphic SCC quotients by induction on the body.

Parallel ($S = S_1 \parallel \cdots \parallel S_n$): dual distributes over parallel, so $dual(S) = dual(S_1) \parallel \cdots \parallel dual(S_n)$. By induction each factor is isomorphic; the product of isomorphic lattices is isomorphic.

Named definitions ($P = S$ **where** $X_1 = S_1, \ldots, X_k = S_k$): $dual(P) = dual(S)$ **where** $X_1 = dual(S_1), \ldots, X_k = dual(S_k)$. Since dual acts on the bodies but preserves the name-reference structure, the same redirections are performed in both $\mathscr{L}(P)$ and $\mathscr{L}(dual(P))$, yielding isomorphic SCC quotients by induction on each S_i. □ □

Proposition 23 says that polarity is invisible at the lattice level: S and $dual(S)$ produce the same lattice structure. Both tools verify $\mathscr{L}(S)/{\equiv} \cong \mathscr{L}(dual(S))/{\equiv}$ for all 108 benchmarks. The Lean 4 mechanisation proves $dual(dual(S)) = S$ on the AST (`Duality.lean`, 156 lines); the state-space isomorphism follows from the constructor-wise argument above.

This result is essential for understanding subtyping.

7 Application: Subtyping as Lattice Embedding

The lattice structure has a concrete consequence for subtype checking.

Definition 24 (Lattice embedding)**.** An *embedding* of a bounded lattice L_1 into L_2 is an injective map $\varphi : L_1 \to L_2$ that preserves and reflects order: $a \leq b$ iff $\varphi(a) \leq \varphi(b)$. Every embedding preserves meets and joins.

Gay and Hole [10] define a coinductive subtyping relation $\leq_{GH}$ on session types. For branch types, $S_1 \leq_{GH} S_2$ when S_1 offers *more* methods (width subtyping) with covariant continuations. For selection types, the subtype offers *fewer* labels (contravariant width) with covariant continuations.

The contravariance of selection width is the *dual* of branch covariance: by Proposition 23, swapping polarity preserves the lattice, so the selection case follows from the branch case applied to the dual types.

Since branch and selection produce *identical state spaces* (Section 3), the embedding relationship depends on which type has more top-level choices—not on the polarity:

Proposition 25 (Width–embedding correspondence)**.** *For all well-formed session types:*

1. *(Branch width.) If* $\&\{m_1{:}S_1, \ldots, m_n{:}S_n\} \leq_{GH} \&\{m_1{:}S'_1, \ldots, m_k{:}S'_k\}$ *with* $k \leq n$*, then* $\mathscr{L}(S_2)/{\equiv}$ *embeds into* $\mathscr{L}(S_1)/{\equiv}$ *(supertype into subtype).*
2. *(Selection width.) If* $\oplus\{l_1{:}S_1, \ldots, l_n{:}S_n\} \leq_{GH} \oplus\{l_1{:}S'_1, \ldots, l_k{:}S'_k\}$ *with* $n \leq k$*, then* $\mathscr{L}(S_1)/{\equiv}$ *embeds into* $\mathscr{L}(S_2)/{\equiv}$ *(subtype into supertype).*

In both cases, the type with fewer top-level choices has the smaller lattice, and it embeds into the type with more choices.

Proof. Gay and Hole [10] define subtyping on recursive types coinductively as the greatest fixpoint of a monotone operator: assume the pair (S_1, S_2) is in the relation, unfold recursive types one step, and check the structural conditions.

Reduction from coinduction to finite induction. The coinductive derivation unfolds recursive types repeatedly, yielding an infinite derivation tree. However, each unfolding $\mu X.S \to S[X := \mu X.S]$ corresponds, at the state-space level, to traversing a back-edge that stays within the same SCC class (Lemma 14). Since the SCC quotients $\mathscr{L}(S_1)/{\equiv}$ and $\mathscr{L}(S_2)/{\equiv}$ are finite DAGs, the derivation visits only finitely many distinct quotient-class pairs. Any derivation step that revisits a pair $([s_1],[s_2])$ already encountered is an implicit appeal to the coinductive hypothesis—on the finite quotient, this corresponds to closing a cycle. Therefore the infinite coinductive derivation factors through a finite set of quotient-class pairs, and we may reason by induction on the DAG structure of the quotient.

Embedding construction. For branch width: $\&\{m_1{:}S_1,\ldots,m_n{:}S_n\} \leq_{GH} \&\{m_1{:}S'_1,\ldots,m_k{:}S'_k\}$ with $k \leq n$ requires $S_i \leq_{GH} S'_i$ for each $i \leq k$ (covariant depth). By induction on the quotient DAG, each $\mathscr{L}(S'_i)/{\equiv}$ embeds into $\mathscr{L}(S_i)/{\equiv}$ via some φ_i. Construct the embedding φ of the supertype lattice into the subtype lattice by: $\varphi([q_\top]) = [q_\top]$, $\varphi([q_\bot]) = [q_\bot]$, and for each quotient class $[s]$ in $\mathscr{L}(S'_i)/{\equiv}$, $\varphi([s]) = \varphi_i([s])$. This is well-defined because the k sub-lattices of the supertype are disjoint above $[q_\bot]$ in the quotient, so the φ_i do not conflict; it is order-preserving and order-reflecting by the inductive hypothesis on each component and the shared extrema. For selection width: the direction is analogous—the subtype (fewer labels) has the smaller state space, and it embeds into the supertype (more labels) by the same construction. □ □

For example, $\&\{m{:}a\,.\,\mathbf{end},\ n{:}b\,.\,\mathbf{end}\} \leq_{GH} \&\{m{:}a\,.\,\mathbf{end}\}$: the supertype's 2-state chain $(q_\top \xrightarrow{m} q_\bot)$ embeds into the subtype's 4-state diamond (Figure 1) via the injection $q_\top \mapsto q_0$, $q_\bot \mapsto q_\bot$. The embedding witnesses that the supertype's protocol is a "sub-lattice" of the subtype's richer protocol.

This gives an order-theoretic characterisation of width subtyping: it reduces to finding a lattice embedding between finite quotient lattices.

Remark 26 (Full Gay–Hole subtyping). Proposition 25 characterises *width* subtyping. The full Gay–Hole relation $\leq_{GH}$ also includes *depth* subtyping: $S_i \leq_{GH} S'_i$ for shared labels. In the lattice picture, each depth step is witnessed by the sub-embedding $\varphi_i : \mathscr{L}(S'_i)/{\equiv} \hookrightarrow \mathscr{L}(S_i)/{\equiv}$ from the proof of Proposition 25. Thus the full relation decomposes as: width determines *which* sub-lattices participate in the embedding, and depth determines *how* corresponding sub-lattices relate. A complete lattice-theoretic characterisation of $\leq_{GH}$ (including the interaction of width and depth with recursion) is the subject of ongoing work.

Both tools implement the embedding check and verify the correspondence for all subtype pairs derivable within each of the 108 benchmark protocols, including 47 with recursion.

8 Empirical Validation

We validate the Reticulate Theorem against a benchmark suite of 108 protocols drawn from three sources:

1. *Literature benchmarks* (34 protocols): Java Iterator, SMTP, HTTP, JDBC Connection, OAuth 2.0, Two-Buyer, and others from the Mungo/StMungo/Scribble literature.
2. *Industrial protocols* (26 protocols): MCP (Model Context Protocol for AI agents), A2A (Google's Agent-to-Agent protocol), TLS Handshake, Saga Orchestrator, WebSocket, gRPC lifecycle.
3. *Synthetic benchmarks* (48 protocols): systematically generated to cover edge cases—deep recursion, wide branching ($n \geq 10$), nested parallel, mixed polarity.

Table 1: Summary of benchmark results. All 108 protocols produce bounded lattices under SCC quotient.

Category	**Protocols**	**Distributive**	**Non-dist.**
Literature	34	32	2
Industrial	26	18	8
Synthetic	48	43	5
Total	**108**	**93**	**15**

Table 2: The 15 non-distributive benchmarks. 13 contain N_5 (pentagon); 3 contain M_3 (diamond).

Protocol	$\|Q\|$	Type	Cause
TLS Handshake	13	N_5	branch-inside-selection (HELLO_RETRY)
Two-Phase Commit	7	N_5	branch-selection nesting
Ki3 Onboarding	32	N_5	nested branch-selection
Ki3 CI/CD Pipeline	41	N_5	4-stage branch-selection nesting
Quantum Measurement	9	N_5	crossed branch-selection
SSH Handshake	8	N_5	branch-selection nesting
Alternative Splicing	7	M_3	three branches sharing outcomes
QCD Gluon Exchange	6	M_3	three-way symmetric reconvergence
Hex (2×2)	10	both	game-tree reconvergence + nesting

Plus 6 synthetic benchmarks with analogous patterns.

Results and distributivity. All 108 protocols produce bounded lattices under SCC quotient (Table 1). We classify each lattice in the Birkhoff hierarchy [2]: 93 (86%) are *distributive* (no forbidden N_5 or M_3 sublattice) and 15 (14%) are non-distributive—13 containing N_5 (pentagon) and 3 containing M_3 (diamond); one protocol (Hex) contains both. All product lattices from parallel composition are distributive, consistent with the lattice-theoretic result that the product of distributive lattices is distributive. This classification is mechanised in Lean 4 with a proof (by `decide`) that the distributive law fails on N_5.

Two structural patterns account for all 15 non-distributive cases. For N_5: *polarity nesting*—a branch as a direct continuation of a selection label, or vice versa—creates incomparable paths through alternating constructors. For M_3: *wide branching*—three or more branches reconverging at a shared $q_\perp$ produces three pairwise-incomparable atoms, the defining configuration of M_3. Binary choice ($n \leq 2$) produces at most M_2 (the 4-element diamond), which is distributive. Characterising precisely which session types yield distributive reticulates remains open.

We additionally enumerate all well-formed terminating session types up to depth 3 with label sets $\{a,b\}$, confirming that every such type produces a lattice—further evidence for universality.

9 Tool Support and Mechanisation

Tool support. We have implemented the state-space construction and lattice checking in two independently developed tools. RETICULATE is a Python 3.12 library and CLI tool. The second tool is a Java 21 annotation processor for compile-time session type checking. Both are thoroughly tested and were developed independently, yet agree on all 108 benchmarks. The lattice check examines all pairs of states for meets and joins—$O(n^2 \cdot m)$ where $n = |Q|$ and $m = |\delta|$—and completes in under 100 ms on commodity

Table 3: Lean 4 mechanisation: one module per paper result. 14 modules (∼2,564 lines), all results in this paper, zero `sorry`.

Paper Result	Lean 4 Module	Lines
Lemma 10 (End)	`EndLemma.lean`	87
Remark 12 (Sequencing=n=1)	`SequencingLemma.lean`	59
Lemma 11 (Choice)	`BranchLemma.lean`	197
Lemma 13 (Absorption)	`BottomAbsorption.lean`	224
Lemma 14 (Recursion)	`RecursionLemma.lean`	475
Lemma 15 (Named defs)	`NamedDefinitions.lean`	266
Lemma 17 (Parallel, n-ary)	`ParallelLemma.lean`	145
Proposition 7 (Extrema)	`UniqueExtrema.lean`	69
Theorem 18 (Reticulate)	`ReticulateTheorem.lean`	118
Corollary 19 (WF-Par)	`Counterexample.lean`	119
Distributivity (Section 8)	`Distributivity.lean`	235
Proposition 23 (Duality)	`Duality.lean`	156
Proposition 25 (Subtyping)	`SubtypingEmbedding.lean`	123
Proposition 21 (Realisability)	`Realizability.lean`	291

hardware for all benchmarks (median 4 ms, maximum 87 ms for the largest benchmark with $|Q| = 41$).

Lean 4 mechanisation. The core proof has been mechanically verified in Lean 4 [19] using Mathlib [24], totalling 14 modules and 2,564 lines of Lean code. All 14 modules listed in Table 3 cover all results in this paper with no `sorry` (all proofs fully discharged). The mechanisation covers all constructor lemmas, the Reticulate Theorem, WF-Par counterexamples, distributivity (N_5), duality, subtyping-embedding, and realisability.

10 Related Work

Session types. Session types were introduced by Honda et al. [12] for binary communication and extended to multiparty settings [13]; Hüttel et al. [14] and Ancona et al. [1] provide comprehensive surveys. Honda et al.'s grammar is a *process* calculus (15 constructs including send/receive, delegation, hiding, and conditional), whereas ours is a *type* grammar (7 constructs). The key type-level correspondences are: label branching $k \triangleright \{l_i : P_i\}$ maps to our branch &; label selection $k \triangleleft l$ maps to our selection $\oplus$; binary parallel $P \mid Q$ on independent processes maps to our n-ary $\|$ on a shared object; and recursive definitions **def** D **in** P map to our named definitions P. Honda et al.'s send/receive and delegation have no counterpart in our object setting, where methods are undirected. Vasconcelos [26] provides a comprehensive foundation. Session types for objects were developed by Vasconcelos et al. [27], Dezani-Ciancaglini et al. [9], and Gay et al. [11], with tool support from MUNGO and STMUNGO [16, 17]. MUNGO uses named definitions as the primary design mechanism; our grammar incorporates the same notation. The key difference is our $\|$ constructor, which has no counterpart in typestate tools. Gay and Hole [10] established subtyping for session types; our Proposition 25 gives width subtyping an order-theoretic characterisation via lattice embeddings. Scalas and Yoshida [22] showed that binary session types can be simplified by reducing to a minimal core. Our work differs in studying the algebraic structure of *state spaces* rather

than the types themselves.

Lattices in programming languages. Lattice theory appears in abstract interpretation [5] (abstract domains) and type hierarchies [2, 8]. We show that *behavioural* types produce lattices at the state-space level, a distinct setting.

Linear logic and session types. Caires and Pfenning [3] established a Curry–Howard correspondence between session types and intuitionistic linear logic: the tensor $\otimes$ and par $\parr$ connectives map to forms of parallel composition, while & and $\oplus$ correspond to our branch and selection. Under this interpretation, the lattice structure we identify on state spaces may connect to the *phase semantics* of linear logic, where denotations form complete lattices. Wadler [28] gave a classical linear logic account ("propositions as sessions"), where negation corresponds to session type duality—the operation that swaps branch and selection, which we showed preserves state-space isomorphism (our duality is an involution on reticulates). Toninho, Caires, and Pfenning [25] extended the correspondence to higher-order sessions; the lattice-theoretic perspective on higher-order types remains open.

Subtyping. Gay and Hole [10] defined subtyping for session types; Dardha et al. [7] revisited it in a pi-calculus setting. Padovani [20] extended subtyping to multiparty session types with a fairness constraint, ensuring that all branches of a selection are eventually exercised; his coinductive characterisation of fair subtyping operates on infinite trees, whereas our lattice embedding operates on finite quotient posets—the two views are complementary. Castagna and Frisch [4] developed semantic subtyping using set-theoretic models. Our characterisation of subtyping as lattice embedding (Section 7) provides an algebraic alternative to the coinductive, fair, and set-theoretic approaches.

Multiparty session types. Honda, Yoshida, and Carbone [13] introduced multiparty session types (MPST), where a global type is projected onto local types for each role. Our Reticulate Theorem applies to local types obtained by projection; whether projection itself is a lattice morphism between global and local reticulates is future work.

Process algebra. Our $\parallel$ operates on a *single shared object* rather than independent processes [18, 21], requiring product lattice construction. Kobayashi [15] and Dardha and Gay [6] use type systems for deadlock-freedom; our lattices complement this by making all interleavings explicit, guaranteeing that every concurrent path reaches $\bot$.

11 Conclusion

We have proved that the state space of every well-formed session type, quotiented by strongly connected components, forms a bounded lattice (Theorem 18). The proof is constructive and modular: each session type constructor preserves lattice structure, with branch and selection creating meets and joins, recursion absorbed by the quotient via top absorption, and n-ary parallel composition yielding n-dimensional product lattices. The grammar is parsimonious—six type constructs plus named definitions for modular protocol design, three n-ary—and the well-formedness conditions reduce to guardedness, termination, and variable independence. We validated the theorem on 108 benchmark protocols using two independently developed, thoroughly tested tools, and mechanically verified the core proof in Lean 4 (14 modules,

all constructor lemmas fully proved). The lattice structure has a concrete application: Gay–Hole width subtyping corresponds to lattice embedding (Proposition 25).

This lattice structure—the *reticulate*—opens several directions. The *distributivity classification* (86% distributive, 14% non-distributive) suggests finer algebraic structure: distributive lattices admit unique prime decompositions (Birkhoff's theorem), which could yield canonical forms for session types. The converse—characterising which bounded lattices arise as reticulates—is addressed by Proposition 21; a complete characterisation remains open. A Galois connection between a program's concrete state space and a session type's reticulate would yield a sound abstract interpretation for protocol conformance [5]. Three grammar extensions preserve lattice structure and are the subject of companion work: first-class sequential composition $S_1 \,.\, S_2$ (a lattice ordinal sum identifying $q^1_\bot$ with $q^2_\top$), synchronous parallel $S_1 \otimes S_2$ (the diagonal sub-lattice of $\mathscr{L}(S_1 \parallel S_2)$), and replication $!S$ (ω-algebraic lattice of the infinite product). Fork-join patterns $(S_1 \parallel S_2) \,.\, S_3$ follow from first-class sequential composition.

Practically, the lattice structure enables algorithms that exploit meets and joins—such as computing the most general common refinement of two protocols (their meet) or the least restrictive generalisation (their join)—operations that are well-defined precisely because the state space is a lattice.

The Reticulate Theorem suggests that session types are more structured than previously recognised. The lattice is not imposed by the theory—it emerges from the syntax. Understanding why may illuminate deeper connections between protocol structure and algebraic order theory.

References

[1] Davide Ancona, Viviana Bono, Mario Bravetti, Joana Campos, Giuseppe Castagna, Pierre-Malo Deniélou, Simon J. Gay, Nils Gesbert, Luca Padovani, António Ravara, Vasco T. Vasconcelos & Nobuko Yoshida (2016): *Behavioral Types in Programming Languages*. *Foundations and Trends in Programming Languages* 3(2–3), pp. 95–230, 10.1561/2500000031.

[2] Garrett Birkhoff (1967): *Lattice Theory*, 3rd edition. *Colloquium Publications* 25, American Mathematical Society.

[3] Luís Caires & Frank Pfenning (2010): *Session Types as Intuitionistic Linear Propositions*. In: *CONCUR 2010 – Concurrency Theory*, *Lecture Notes in Computer Science* 6269, Springer, pp. 222–236, 10.1007/978-3-642-15375-4_16.

[4] Giuseppe Castagna & Alain Frisch (2005): *A Gentle Introduction to Semantic Subtyping*. In: *Proceedings of the 7th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming (PPDP)*, ACM, pp. 198–208, 10.1145/1069774.1069793.

[5] Patrick Cousot & Radhia Cousot (1977): *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. In: *Proceedings of the 4th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, pp. 238–252, 10.1145/512950.512973.

[6] Ornela Dardha & Simon J. Gay (2018): *A New Linear Logic for Deadlock-Free Session-Typed Processes*. In: *Foundations of Software Science and Computation Structures (FoSSaCS)*, Springer, pp. 91–109, 10.1007/978-3-319-89366-2_5.

[7] Ornela Dardha, Elena Giachino & Davide Sangiorgi (2017): *Session Types Revisited*. *Information and Computation* 256, pp. 253–286, 10.1016/j.ic.2017.06.002.

[8] B. A. Davey & H. A. Priestley (2002): *Introduction to Lattices and Order*, 2nd edition. Cambridge University Press, 10.1017/CBO9780511809088.

[9] Mariangiola Dezani-Ciancaglini, Dimitris Mostrous, Nobuko Yoshida & Sophia Drossopoulou (2006): *Session Types for Object-Oriented Languages*. In: *European Conference on Object-Oriented Programming (ECOOP)*, Springer, pp. 328–352, 10.1007/11785477_20.

[10] Simon J. Gay & Malcolm Hole (2005): *Subtyping for Session Types in the Pi Calculus*. *Acta Informatica* 42(2–3), pp. 191–225, 10.1007/s00236-005-0177-z.

[11] Simon J. Gay, Vasco T. Vasconcelos, António Ravara, Nils Gesbert & Alexandre Zua Caldeira (2010): *Modular Session Types for Distributed Object-Oriented Programming*. In: *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, pp. 299–312, 10.1145/1706299.1706335.

[12] Kohei Honda, Vasco T. Vasconcelos & Makoto Kubo (1998): *Language Primitives and Type Discipline for Structured Communication-Based Programming*. In: *European Symposium on Programming (ESOP)*, *Lecture Notes in Computer Science* 1381, Springer, pp. 122–138, 10.1007/BFb0053567.

[13] Kohei Honda, Nobuko Yoshida & Marco Carbone (2008): *Multiparty Asynchronous Session Types*. In: *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, pp. 273–284, 10.1145/1328438.1328472.

[14] Hans Hüttel, Ivan Lanese, Vasco T. Vasconcelos, Luís Caires, Marco Carbone, Pierre-Malo Deniélou, Dimitris Mostrous, Luca Padovani, António Ravara, Emilio Tuosto, Hugo Torres Vieira & Gianluigi Zavattaro (2016): *Foundations of Session Types and Behavioural Contracts*. *ACM Computing Surveys* 49(1), pp. 3:1–3:36, 10.1145/2873052.

[15] Naoki Kobayashi (2006): *A New Type System for Deadlock-Free Processes*. In: *CONCUR*, Springer, pp. 233–247, 10.1007/11817949_16.

[16] Dimitrios Kouzapas, Ornela Dardha, Roly Perera & Simon J. Gay (2016): *Typechecking Protocols with Mungo and StMungo*. In: *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming (PPDP)*, ACM, pp. 146–159, 10.1145/2967973.2968595.

[17] Dimitrios Kouzapas, Ornela Dardha, Roly Perera & Simon J. Gay (2017): *Typechecking Protocols with Mungo and StMungo: A Session Type Toolchain for Java*. In: *Behavioural Types: from Theory to Tools*, River Publishers, pp. 371–395.

[18] Robin Milner (1989): *Communication and Concurrency*. Prentice Hall.

[19] Leonardo de Moura & Sebastian Ullrich (2021): *The Lean 4 Theorem Prover and Programming Language*. In: *Automated Deduction – CADE 28*, *Lecture Notes in Computer Science* 12699, Springer, pp. 625–635, 10.1007/978-3-030-79876-5_37.

[20] Luca Padovani (2016): *Fair Subtyping for Multi-party Session Types*. *Mathematical Structures in Computer Science* 26(3), pp. 424–464, 10.1017/S096012951400022X.

[21] Davide Sangiorgi & David Walker (2001): *The π-Calculus: A Theory of Mobile Processes*. Cambridge University Press.

[22] Alceste Scalas & Nobuko Yoshida (2019): *Less Is More: Multiparty Session Types Revisited*. *Proceedings of the ACM on Programming Languages* 3(POPL), pp. 30:1–30:29, 10.1145/3290343.

[23] Robert Tarjan (1972): *Depth-First Search and Linear Graph Algorithms*. *SIAM Journal on Computing* 1(2), pp. 146–160, 10.1137/0201010.

[24] The mathlib Community (2020): *The Lean Mathematical Library*. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pp. 367–381, 10.1145/3372885.3373824.

[25] Bernardo Toninho, Luís Caires & Frank Pfenning (2013): *Higher-Order Processes, Functions, and Sessions: A Monadic Integration*. In: *European Symposium on Programming (ESOP)*, *Lecture Notes in Computer Science* 7792, Springer, pp. 350–369, 10.1007/978-3-642-37036-6_20.

[26] Vasco T. Vasconcelos (2012): *Fundamentals of Session Types*. *Information and Computation* 217, pp. 52–70, 10.1016/j.ic.2012.05.002.

[27] Vasco T. Vasconcelos, Simon J. Gay & António Ravara (2006): *Type Checking a Multithreaded Functional Language with Session Types*. Theoretical Computer Science 368(1–2), pp. 64–87, 10.1016/j.tcs.2006.06.028.

[28] Philip Wadler (2012): *Propositions as Sessions*. In: *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, ACM, pp. 273–286, 10.1145/2364527.2364568.